

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №7
По СПП

Выполнил:
студент 3 курса
группы ПО-12
Коханская Е.В.
Проверил:
Кулик А. Д.

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

"""

Лабораторная работа №7

Вариант 8

Задание 1: Движение строк по экрану

Задание 2: Фрактал "Кривая дракона"

"""

import random

import tkinter as tk

from tkinter import ttk, colorchooser, messagebox

from datetime import datetime

from typing import List, Tuple

from dataclasses import dataclass

=====

ЗАДАНИЕ 1: ДВИЖЕНИЕ СТРОК ПО ЭКРАНУ

=====

@dataclass

class MovingText:

"""Класс для движущейся строки текста."""

text: str

x: float

y: float

dx: float

dy: float

color: str

font_size: int

class TextAnimation:

```
"""Класс для анимации движущихся строк."""
```

```
def __init__(self, canvas: tk.Canvas, strings: List[str]):
```

```
    """
```

```
    Инициализация анимации.
```

```
    Args:
```

```
        canvas: Canvas для рисования
```

```
        strings: Список строк для анимации
```

```
    """
```

```
    self.canvas = canvas
```

```
    self.strings = strings
```

```
    self.moving_texts: List[dict] = []
```

```
    self.running = False
```

```
    self.after_id = None
```

```
    self.current_index = 0
```

```
    self.speed = 5
```

```
    self.canvas_width = 0
```

```
    self.canvas_height = 0
```

```
def set_canvas_size(self, width: int, height: int) -> None:
```

```
    """Установка размера canvas."""
```

```
    self.canvas_width = width
```

```
    self.canvas_height = height
```

```
def start(self) -> None:
```

```
    """Запуск анимации."""
```

```
    self.running = True
```

```
    self.current_index = 0
```

```
    self._add_next_text()
```

```
def stop(self) -> None:
```

```
    """Остановка анимации."""
```

```
    self.running = False
```

```

if self.after_id:

    self.canvas.after_cancel(self.after_id)

    self.after_id = None


def clear(self) -> None:

    """Очистка всех текстов."""

    for text in self.moving_texts:

        self.canvas.delete(text["id"])

    self.moving_texts.clear()


def set_speed(self, speed: int) -> None:

    """Установка скорости движения."""

    self.speed = max(1, min(20, speed))


def _add_next_text(self) -> None:

    """Добавление нового текста на экран."""

    if not self.running:

        return

    if self.current_index < len(self.strings):

        text = self.strings[self.current_index]

        self._create_moving_text(text)

        self.current_index += 1

    else:

        self.current_index = 0

    if self.running:

        self.after_id = self.canvas.after(2000, self._add_next_text)


def _create_moving_text(self, text: str) -> None:

    """

    Создание нового движущегося текста.

    Args:

```

text: Текст для отображения

"""

dx = random.choice([-self.speed, self.speed])

dy = random.choice([-self.speed, self.speed])

side = random.choice(["top", "bottom", "left", "right"])

if side == "top":

 x = random.randint(50, self.canvas_width - 50)

 y = 20

 if dy < 0:

 dy = -dy

elif side == "bottom":

 x = random.randint(50, self.canvas_width - 50)

 y = self.canvas_height - 20

 if dy > 0:

 dy = -dy

elif side == "left":

 x = 20

 y = random.randint(50, self.canvas_height - 50)

 if dx < 0:

 dx = -dx

else:

 x = self.canvas_width - 20

 y = random.randint(50, self.canvas_height - 50)

 if dx > 0:

 dx = -dx

colors = [

 "red",

 "blue",

 "green",

 "orange",

 "purple",

 "cyan",

```

        "magenta",
        "yellow",
    ]
    color = random.choice(colors)
    font_size = random.randint(14, 36)

    text_id = self.canvas.create_text(
        x,
        y,
        text=text,
        font=("Arial", font_size, "bold"),
        fill=color,
        anchor="center",
    )

    self.moving_texts.append(
        {
            "id": text_id,
            "text": text,
            "x": x,
            "y": y,
            "dx": dx,
            "dy": dy,
            "color": color,
            "font_size": font_size,
        }
    )

def update(self) -> None:
    """Обновление позиций всех текстов."""
    if not self.running:
        return

    for text in self.moving_texts:

```

```
text["x"] += text["dx"]
```

```
text["y"] += text["dy"]
```

```
if text["x"] - 50 <= 0 or text["x"] + 50 >= self.canvas_width:
```

```
    text["dx"] = -text["dx"]
```

```
if text["y"] - 20 <= 0 or text["y"] + 20 >= self.canvas_height:
```

```
    text["dy"] = -text["dy"]
```

```
self.canvas.coords(text["id"], text["x"], text["y"])
```

```
if self.running:
```

```
    self.after_id = self.canvas.after(50, self.update)
```

```
# =====
```

```
# ЗАДАНИЕ 2: ФРАКТАЛ "КРИВАЯ ДРАКОНА"
```

```
# =====
```

```
class DragonCurve:
```

```
    """Класс для генерации и отображения фрактала "Кривая дракона"."""
```

```
    def __init__(self, canvas: tk.Canvas):
```

```
        """
```

```
        Инициализация фрактала.
```

```
        Args:
```

```
            canvas: Canvas для рисования
```

```
        """
```

```
        self.canvas = canvas
```

```
        self.order = 5
```

```
        self.line_color = "blue"
```

```
        self.line_width = 2
```

```
        self.points: List[Tuple[float, float]] = []
```

```
    def set_order(self, order: int) -> None:
```

```

        """Установка порядка фрактала."""

        self.order = max(1, min(15, order))

def set_color(self, color: str) -> None:

    """Установка цвета линии."""

    self.line_color = color

def set_line_width(self, width: int) -> None:

    """Установка толщины линии."""

    self.line_width = max(1, min(10, width))

def generate(self, start_x: float, start_y: float, size: float) -> None:

    """

    Генерация кривой дракона.

    Args:

        start_x: Начальная X координата

        start_y: Начальная Y координата

        size: Размер сегмента

    """

    sequence = self._generate_sequence(self.order)

    self.points = self._calculate_points(sequence, start_x, start_y, size)

def _generate_sequence(self, order: int) -> List[str]:

    """

    Генерация последовательности поворотов.

    Args:

        order: Порядок фрактала

    Returns:

        Список поворотов ('L' или 'R')

    """

    if order == 0:

```

```

        return []

    if order == 1:
        return ["R"]

    prev = self._generate_sequence(order - 1)
    result = prev + ["R"] + self._reverse_complement(prev)
    return result

```

```
def _reverse_complement(self, seq: List[str]) -> List[str]:
```

```
    """
```

Получение обратной комплементарной последовательности.

Args:

seq: Исходная последовательность

Returns:

Обратная комплементарная последовательность

```
    """
```

```
    complement = {"L": "R", "R": "L"}
```

```
    reversed_seq = seq[::-1]
```

```
    return [complement[x] for x in reversed_seq]
```

```
def _calculate_points(
```

```
    self, sequence: List[str], start_x: float, start_y: float, size: float
```

```
) -> List[Tuple[float, float]]:
```

```
    """
```

Вычисление точек кривой.

Args:

sequence: Последовательность поворотов

start_x: Начальная X

start_y: Начальная Y

size: Длина сегмента

Returns:

Список точек кривой

"""

```
points = [(start_x, start_y)]
```

```
direction = 0
```

```
for move in sequence:
```

```
    x, y = points[-1]
```

```
    if direction == 0:
```

```
        x += size
```

```
    elif direction == 1:
```

```
        y += size
```

```
    elif direction == 2:
```

```
        x -= size
```

```
    else:
```

```
        y -= size
```

```
    points.append((x, y))
```

```
if move == "L":
```

```
    direction = (direction + 1) % 4
```

```
else:
```

```
    direction = (direction - 1) % 4
```

```
return points
```

```
def draw(self) -> None:
```

```
    """Отрисовка кривой дракона."""
```

```
    if not self.points:
```

```
        return
```

```
    for i in range(len(self.points) - 1):
```

```
        x1, y1 = self.points[i]
```

```
        x2, y2 = self.points[i + 1]
```

```
        self.canvas.create_line(
```

```
        x1,  
        y1,  
        x2,  
        y2,  
        fill=self.line_color,  
        width=self.line_width,  
        tags="fractal",  
    )
```

```
def clear(self) -> None:
```

```
    """Очистка фрактала с canvas."""  
    self.canvas.delete("fractal")  
    self.points.clear()
```

```
# =====  
# ГЛАВНОЕ ПРИЛОЖЕНИЕ  
# =====
```

```
class FractalApp:
```

```
    """Главное приложение с вкладками для двух заданий."""
```

```
def __init__(self):
```

```
    """Инициализация приложения."""
```

```
    self.root = tk.Tk()  
    self.root.title("Лабораторная работа №7 - Вариант 8")  
    self.root.geometry("1000x700")  
    self.root.resizable(True, True)
```

```
    self.text_animation = None
```

```
    self.dragon_curve = None
```

```
    self.notebook = ttk.Notebook(self.root)
```

```
    self.notebook.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
```

```
self.text_tab = ttk.Frame(self.notebook)

self.notebook.add(self.text_tab, text="Движение строк")


self.fractal_tab = ttk.Frame(self.notebook)

self.notebook.add(self.fractal_tab, text="Кривая дракона")


self._init_text_tab()

self._init_fractal_tab()
```

```
def _init_text_tab(self) -> None:
```

```
    """Инициализация вкладки с движением строк."""

    control_frame = ttk.Frame(self.text_tab)

    control_frame.pack(fill=tk.X, padx=10, pady=5)


    ttk.Label(control_frame, text="Строки (через запятую):").grid(
        row=0, column=0, sticky=tk.W
    )


    self.texts_entry = tk.Text(control_frame, height=5, width=50)

    self.texts_entry.grid(row=1, column=0, columnspan=4, pady=5, sticky=tk.W)

    default_texts = (
        "Привет, мир!,Python,Фракталы,Программирование,"
        "Дракон,Анимация,Tkinter,Лабораторная работа"
    )

    self.texts_entry.insert("1.0", default_texts)


    ttk.Label(control_frame, text="Скорость:").grid(
        row=2, column=0, sticky=tk.W, padx=5
    )


    self.speed_scale = ttk.Scale(
        control_frame, from_=1, to=20, orient=tk.HORIZONTAL, length=200
    )

    self.speed_scale.set(5)

    self.speed_scale.grid(row=2, column=1, sticky=tk.W)
```

```

self.speed_label = ttk.Label(control_frame, text="5")
self.speed_label.grid(row=2, column=2, padx=5)

def update_speed_label(value):
    self.speed_label.configure(text=str(int(float(value))))

self.speed_scale.configure(command=update_speed_label)

button_frame = ttk.Frame(control_frame)
button_frame.grid(row=2, column=3, padx=20)

tk.Button(button_frame, text="Старт", command=self._start_text_animation).pack(
    side=tk.LEFT, padx=5
)
tk.Button(button_frame, text="Стоп", command=self._stop_text_animation).pack(
    side=tk.LEFT, padx=5
)
tk.Button(
    button_frame, text="Очистить", command=self._clear_text_animation
).pack(side=tk.LEFT, padx=5)
tk.Button(
    button_frame,
    text="Скриншот",
    command=lambda: self._take_screenshot(self.text_canvas),
).pack(side=tk.LEFT, padx=5)

canvas_frame = ttk.Frame(self.text_tab)
canvas_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

self.text_canvas = tk.Canvas(
    canvas_frame, bg="white", highlightthickness=1, highlightbackground="gray"
)
self.text_canvas.pack(fill=tk.BOTH, expand=True)

```

```

self.text_canvas.bind("<Configure>", self._on_text_canvas_resize)

def _init_fractal_tab(self) -> None:
    """Инициализация вкладки с фракталом."""
    control_frame = ttk.Frame(self.fractal_tab)
    control_frame.pack(fill=tk.X, padx=10, pady=5)

    ttk.Label(control_frame, text="Порядок (1-15):").grid(
        row=0, column=0, sticky=tk.W, padx=5
    )
    self.order_spinbox = ttk.Spinbox(control_frame, from_=1, to=15, width=10)
    self.order_spinbox.set(5)
    self.order_spinbox.grid(row=0, column=1, sticky=tk.W, padx=5)

    ttk.Label(control_frame, text="Толщина линии:").grid(
        row=0, column=2, sticky=tk.W, padx=5
    )
    self.width_spinbox = ttk.Spinbox(control_frame, from_=1, to=10, width=10)
    self.width_spinbox.set(2)
    self.width_spinbox.grid(row=0, column=3, sticky=tk.W, padx=5)

    ttk.Button(
        control_frame, text="Выбрать цвет", command=self._choose_fractal_color
    ).grid(row=0, column=4, padx=10)
    self.color_label = ttk.Label(control_frame, text="синий", foreground="blue")
    self.color_label.grid(row=0, column=5, padx=5)

    ttk.Button(control_frame, text="Построить", command=self._draw_fractal).grid(
        row=0, column=6, padx=10
    )
    ttk.Button(control_frame, text="Очистить", command=self._clear_fractal).grid(
        row=0, column=7, padx=5
    )
    ttk.Button(

```

```

        control_frame,

        text="Скриншот",

        command=lambda: self._take_screenshot(self.fractal_canvas),

    ).grid(row=0, column=8, padx=5)


    canvas_frame = ttk.Frame(self.fractal_tab)

    canvas_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)


    self.fractal_canvas = tk.Canvas(

        canvas_frame, bg="white", highlightthickness=1, highlightbackground="gray"

    )

    self.fractal_canvas.pack(fill=tk.BOTH, expand=True)


    self.fractal_canvas.bind("<Configure>", self._on_fractal_canvas_resize)


    self.dragon_curve = DragonCurve(self.fractal_canvas)


def _on_text_canvas_resize(self, event: tk.Event) -> None:

    """Обработчик изменения размера canvas."""

    if self.text_animation:

        self.text_animation.set_canvas_size(event.width, event.height)


def _on_fractal_canvas_resize(self, event: tk.Event) -> None:

    """Обработчик изменения размера canvas."""

    # Пустой метод, но он нужен для привязки события

    # Фрактал будет перестроен при нажатии кнопки "Построить"


def _start_text_animation(self) -> None:

    """Запуск анимации строк."""

    texts_str = self.texts_entry.get("1.0", tk.END).strip()

    strings = [s.strip() for s in texts_str.split(",") if s.strip()]

    if not strings:

        messagebox.showwarning("Предупреждение", "Введите хотя бы одну строку!")

```

```
return
```

```
self._stop_text_animation()
```

```
self.text_animation = TextAnimation(self.text_canvas, strings)
```

```
width = self.text_canvas.winfo_width()
```

```
height = self.text_canvas.winfo_height()
```

```
self.text_animation.set_canvas_size(width, height)
```

```
self.text_animation.set_speed(int(self.speed_scale.get()))
```

```
self.text_animation.start()
```

```
self.text_animation.update()
```

```
def _stop_text_animation(self) -> None:
```

```
    """Остановка анимации строк."""
```

```
    if self.text_animation:
```

```
        self.text_animation.stop()
```

```
        self.text_animation = None
```

```
def _clear_text_animation(self) -> None:
```

```
    """Очистка анимации строк."""
```

```
    self._stop_text_animation()
```

```
    self.text_canvas.delete("all")
```

```
def _choose_fractal_color(self) -> None:
```

```
    """Выбор цвета для фрактала."""
```

```
    color = colorchooser.askcolor(title="Выберите цвет линии")[1]
```

```
    if color:
```

```
        self.color_label.configure(text=color, foreground=color)
```

```
        self.dragon_curve.set_color(color)
```

```
def _draw_fractal(self) -> None:
```

```
    """Построение фрактала."""
```

```
    self.dragon_curve.clear()
```

```

order = int(self.order_spinbox.get())

width_val = int(self.width_spinbox.get())

color = self.color_label.cget("text")


self.dragon_curve.set_order(order)

self.dragon_curve.set_line_width(width_val)

self.dragon_curve.set_color(color)


canvas_width = self.fractal_canvas.winfo_width()
canvas_height = self.fractal_canvas.winfo_height()


if canvas_width <= 1:
    canvas_width = 800
    canvas_height = 500


size = min(canvas_width, canvas_height) / (2 ** (order / 2 + 1))
start_x = canvas_width // 2 - (size * (2**order) / 4)
start_y = canvas_height // 2


self.dragon_curve.generate(start_x, start_y, size)

self.dragon_curve.draw()


def _clear_fractal(self) -> None:
    """Очистка фрактала."""
    self.dragon_curve.clear()


def _take_screenshot(self, canvas: tk.Canvas) -> None:
    """
    Создание скриншота canvas.

    Args:
        canvas: Canvas для сохранения
    """
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

```

```
filename = f"screenshot_{timestamp}.ps"
```

```
try:
```

```
    canvas.postscript(file=filename, colormode="color")
```

```
    messagebox.showinfo("Скриншот", f"Скриншот сохранён как {filename}")
```

```
except (OSError, IOError) as e:
```

```
    messagebox.showerror("Ошибка", f"Не удалось сохранить скриншот: {e}")
```

```
def run(self) -> None:
```

```
    """Запуск приложения."""
```

```
    self.root.protocol("WM_DELETE_WINDOW", self._on_closing)
```

```
    self.root.mainloop()
```

```
def _on_closing(self) -> None:
```

```
    """Обработчик закрытия окна."""
```

```
    self._stop_text_animation()
```

```
    self.root.destroy()
```

```
def main():
```

```
    """Главная функция."""
```

```
    app = FractalApp()
```

```
    app.run()
```

```
if __name__ == "__main__":
```

```
    main()
```

